

B2 SQL AVANCÉ

Délio Duchesne-Brajon

Groupe 1

1/ La gestion des jointures

1)

2)

3)

1.1/ Jointure Interne

4)

5)

1.2/ Jointure Externe

6)

1.3 / Plus loin avec les jointures

7)

8)

9)

2/ Les vues

3/ Les “multi” requêtes (UNION,MINUS)

3.1/ UNION

10)

11)

12)

13)

4.2/ Exemples

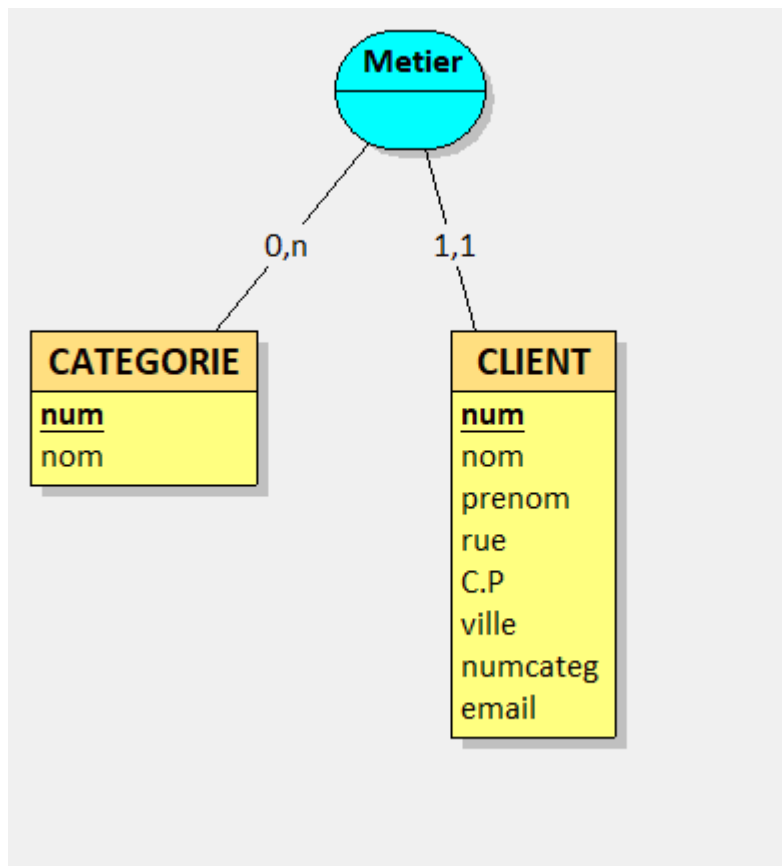
14)

15)

16)

1/ La gestion des jointures

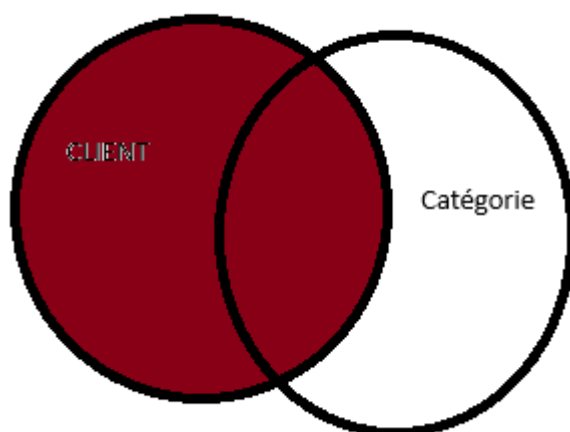
1)



2)

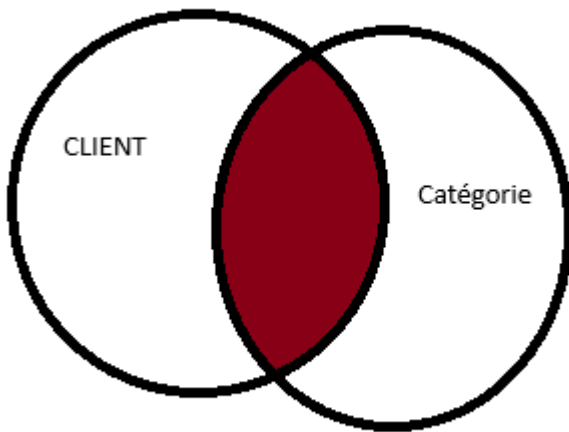
```
SELECT * FROM CLIENT, CATEGORIE WHERE CLIENT.numcateg = CATEGORIE.num
```

3)



1.1/ Jointure Interne

4)

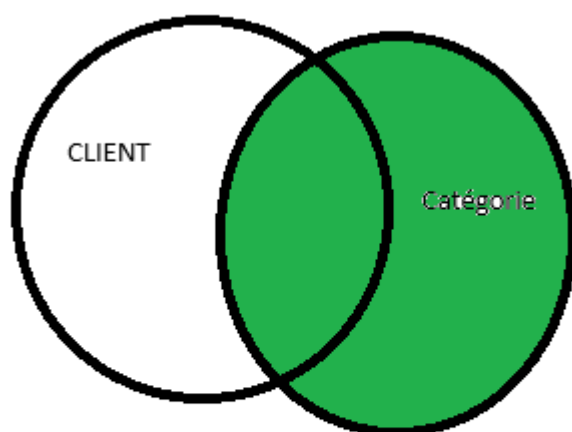
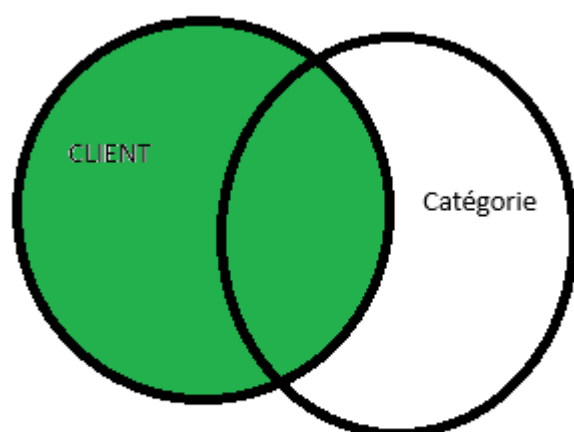


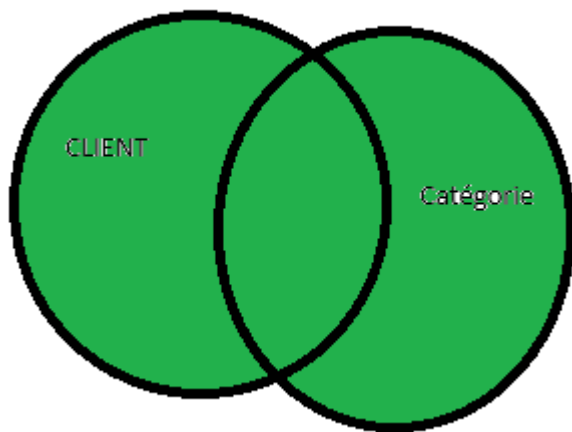
5)

```
SELECT * FROM CLIENT INNER JOIN CATEGORIE ON CLIENT.numcateg =  
CATEGORIE.num;
```

1.2/ Jointure Externe

6)

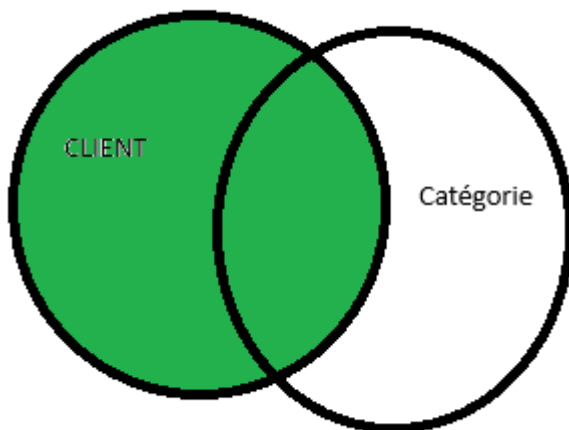




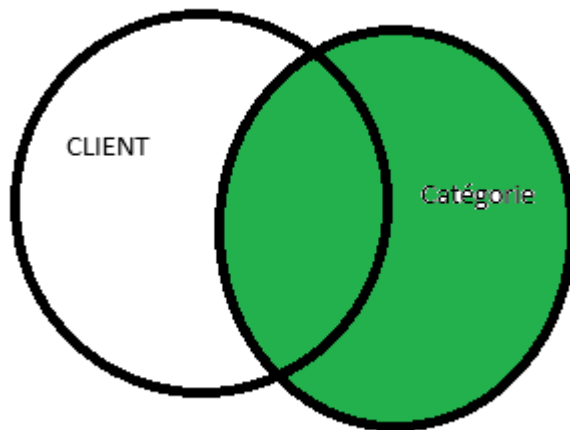
1.3 / Plus loin avec les jointures

7)

```
SELECT * AS nom_categorie  
FROM CLIENT  
LEFT JOIN CATEGORIE ON CLIENT.numcateg = CATEGORIE.num;
```



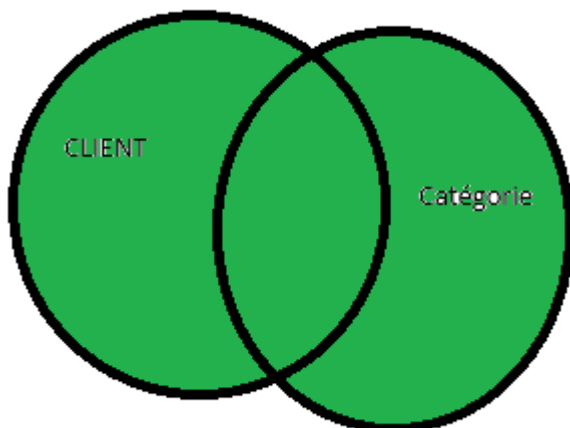
8)



celles qui ne sont rattachés à aucun client.

Affichez toutes les catégories mêmes

9)



Le schéma d'un FULL OUTER JOIN représente l'union totale de vos deux tables. C'est la vision la plus large possible de vos données.

2/ Les vues

3/ Les “multi” requêtes (UNION,MINUS)

3.1/ UNION

10)

```
SELECT * FROM ELEVE INNER JOIN UTILISATEUR ON Num_user = Num;
```

11)

```
SELECT * FROM PROF  
INNER JOIN UTILISATEUR ON NUM_USER = Num;
```

12)

```
SELECT UTILISATEUR.*, PROF.matiere, ELEVE.option  
FROM UTILISATEUR  
LEFT JOIN PROF ON NUM_USER = Num;  
LEFT JOIN ELEVE ON Num_user = Num;
```

13)

```
CREATE VIEW user  
AS  
SELECT utilisateur.*,  
       eleve.option,  
       prof.matiere,  
CASE  
  WHEN eleve.num_user IS NOT NULL THEN 1  
  WHEN prof.num_user IS NOT NULL THEN 2  
  ELSE 0  
END AS type  
FROM utilisateur LEFT JOIN eleve ON utilisateur.num = eleve.num_user LEFT JOIN prof  
ON utilisateur.num = prof.num_user;
```

4.2/ Exemples

14)

15)

16)